PHASE

1

Build the Secure World

OP-TEE OS

WHAT CHANGED

- **Target selected**
  Raspberry Pi 3, AArch64
- **Secure OS compiled**

  Cross-built on the laptop
- **Boot payload prepared**

  TF-A-compatible split images

OUTPUT TREE

generated / verified files

```
optee_os/
`-- out/arm/core/
    |-- tee.bin           [Secure World]
    |-- tee.elf           [debug image]
    |-- tee-header_v2.bin [TF-A header]
    |-- tee-pager_v2.bin  [pager payload]
    `-- tee-pageable_v2.bin

Loaded by TF-A as BL32
```

OUTCOME **Secure World binaries are available.**

BOUNDARY Compilation only; not booted.

COMPLETE

PHASE

2

Package the secure boot chain

TF-A + U-Boot

WHAT CHANGED

- **U-Boot built**
  Normal World payload: BL33
- **TF-A configured**

  OP-TEE dispatcher + BL32
- **Boot image packaged**

  BL1 and FIP combined

OUTPUT TREE

generated / verified files

```
u-boot/
|-- u-boot.bin       [BL33]
|-- u-boot             [ELF/debug]
`-- System.map        [symbols]

trusted-firmware-a/
`-- build/rpi3/debug/
    |-- bl1.bin          [early stage]
    |-- fip.bin          [BL32 + BL33]
    `-- armstub8.bin     [BL1 + FIP]
```

OUTCOME **Secure and Normal payloads are packaged.**

BOUNDARY Packaging only; not booted.

COMPLETE

PHASE

3

Prepare the Normal World kernel

Linux + standard DTBs

WHAT CHANGED

- **Kernel configured CONFIG_TEE=y, CONFIG_OPTEE=y**
- **ARM64 Linux compiled**

  Raspberry Pi 3 target
- **Existing DTS compiled**

  No new hardware or custom node

OUTPUT TREE

generated / verified files

```
Step3_Output/
|-- Image                    [ARM64 kernel]
|-- linux.config             [kernel config]
`-- dtbs/
    |-- bcm2710-rpi-3-b.dtb       [Model B]
    `-- bcm2710-rpi-3-b-plus.dtb  [Model B+]

Loaded by U-Boot with Image
```

OUTCOME **Kernel and board descriptions are verified.**

BOUNDARY Communication not yet proven.

COMPLETE

PHASE

4

Create a reproducible build base

Verify + preserve

WHAT CHANGED

- **Authoritative tree selected**
  Laptop optee_full
- **Versions compared**

  Laptop and Pi revisions match
- **Evidence preserved**

  Backups + SHA-256 checksums

OUTPUT TREE

generated / verified files

```
Step4_Verification/
|-- STEP4_COMPLETION.md
`-- verified_artifacts/
    |-- optee/
    |   `-- tee*.bin + tee.elf
    |-- tf-a/
    |   `-- armstub8.bin + fip.bin
    |-- u-boot/
    |   `-- u-boot.bin
    `-- linux/
        `-- Image + DTBs + config
```

OUTCOME **One controlled, checksummed workspace.**

BOUNDARY Quality control; no runtime feature.

COMPLETE

PHASE

5

Build the Linux user space

Buildroot + OP-TEE runtime

WHAT CHANGED

- Configured a minimal ARM64 Buildroot system
- Generated the Linux root filesystem
- Integrated the OP-TEE Normal World runtime
- Added test clients, examples and signed TAs

OUTPUT TREE

generated / verified files

```
out-br/
├── images/rootfs.ext2
└── target/
    ├── bin/, sbin/, etc/
    ├── usr/lib/libteec.so
    ├── usr/sbin/tee-supplicant
    ├── usr/bin/xtest
    ├── usr/bin/optee_example_*
    └── lib/optee_armtz/*.ta
```

OUTCOME **Boot-ready ARM64 root filesystem with the OP-TEE user-space environment.**

BOUNDARY Created here; packaged into the SD image in Step 7 and verified in Steps 8–9.

COMPLETE

PHASE

6

COMPLETE

Assemble the Raspberry Pi boot files

Boot-chain assembly

WHAT CHANGED

- Collected all Raspberry Pi boot files
- Built armstub8.bin from TF-A BL1 + FIP
- FIP contains BL2, BL31, OP-TEE BL32 and U-Boot BL33
- Added Linux, DTBs and the U-Boot environment
- Defined the handoff from firmware to Linux

OUTPUT TREE

generated / verified files

```
optee_full/out/boot/
├── bootcode.bin + start*.elf + fixup*.dat
│   └── Raspberry Pi GPU firmware
├── config.txt                [boot selection]
├── armstub8.bin
│   ├── TF-A BL1
│   └── FIP
│       ├── TF-A BL2
│       ├── TF-A BL31         [EL3 monitor]
│       ├── OP-TEE BL32        [Secure World]
│       └── U-Boot BL33        [Normal World]
├── kernel8.img               [Linux kernel]
├── bcm2710-*.dtb             [hardware data]
└── uboot.env                 [kernel handoff]

BOOT: Firmware → BL1 → BL2 → BL31 → BL32
      → BL33/U-Boot → Linux
```

OUTCOME **One connected boot payload with a defined Secure-to-Normal World handoff.**

BOUNDARY Files assembled here; the boot and rootfs partitions are packaged in Step 7.

PHASE

7

COMPLETE

Generate and verify the SD-card image

Two-partition image

WHAT CHANGED

- Created one MBR SD-card image
- Partition 1 received the Step 6 boot payload
- Partition 2 received the Step 5 Buildroot rootfs
- Flashed and compared the 8 GB test card
- Confirmed the written files matched the image

OUTPUT TREE

generated / verified files

```
rpi3-sdcard.img                    [MBR disk]
├── Partition 1: FAT16 / boot      [~63 MiB]
│   ├── Raspberry Pi firmware
│   ├── config.txt
│   ├── armstub8.bin
│   │   └── BL1 + FIP(BL2/BL31/BL32/BL33)
│   ├── kernel8.img + DTBs
│   └── uboot.env
└── Partition 2: ext4 / rootfs   [~256 MiB]
    ├── bin/, sbin/, etc/         [Buildroot]
    ├── lib/modules/              [Linux modules]
    ├── usr/lib/libteec.so
    ├── usr/sbin/tee-supplicant
    ├── usr/bin/xtest + examples
    └── lib/optee_armtz/*.ta      [signed TAs]

Image write verification: MATCH
```

OUTCOME **One flashable image containing both the boot chain and Linux root filesystem.**

BOUNDARY Packaging completed here; physical boot and runtime proof follow in Steps 8–9.

PHASE

8

Boot the custom image on Raspberry Pi 3

Hardware boot proof

WHAT I DID

- **Flashed rpi3-sdcard.img to the 8 GB test card**
- **Booted Raspberry Pi 3 from the custom SD image**
- **Reached the Buildroot root shell over HDMI**
- **Confirmed AArch64 Linux and four CPU cores**
- **Obtained an Ethernet address and connected by SSH**

BOOT EVIDENCE

observed on the real

```
rpi3-sdcard.img
       ↓  dd to /dev/sda
8 GB SD card → Raspberry Pi 3
       ↓
Firmware → Secure boot chain → Linux
       ↓
Buildroot login and root shell
       ↓
eth0 DHCP → SSH access

$ whoami       → root
$ uname -m     → aarch64
$ ip -4 addr   → 10.237.27.131/23
```

OUTCOME **The custom image booted on real hardware and was accessible locally and remotely.**

BOUNDARY HDMI/SSH boot proved; an early-stage UART capture still requires a USB-to-TTL adapter.

PARTIAL

PHASE

9

Prove Linux-to-OP-TEE communication

Secure runtime proof

VERIFICATION RESULTS

- **Linux detected OP-TEE revision 4.10**
- **OP-TEE kernel driver initialized successfully**
- **/dev/tee0 and /dev/teepriv0 were created**
- **tee-supplicant started automatically**
- **xtest completed with zero failures**

COMMUNICATION + PROOF

Normal World to

```
Normal World
xtest → libteec → /dev/tee0
          ↓ Linux OP-TEE driver
          ↓ SMC via TF-A BL31
Secure World
OP-TEE BL32 → signed Trusted Application

$ dmesg | grep -i optee
  revision 4.10; initialized driver
$ ls /dev/tee*
  /dev/tee0  /dev/teepriv0
$ xtest
  114 cases | 36,114 subtests
  0 failed   | 0 skipped
```

OUTCOME **Linux successfully invoked OP-TEE services and executed signed TAs in Secure World.**

BOUNDARY The generic OP-TEE platform passed; the project-specific secure application follows.

COMPLETE

PHASE

10

COMPLETE

Build and run the project Trusted App

secure_can_demo

WHAT CHANGED

- Project client created
  TEEC session + command invocation
- Signed AArch64 TA created
  Unique UUID and strict parameters
- Real Pi round trip passed
  Challenge, response and SCAN marker

OUTPUT TREE

generated / verified files

```
secure_can_demo [Normal World]
        |  challenge 0x12345678
        v
TEEC_InvokeCommand
        |  TrustZone boundary
        v
Secure CAN TA [Secure World]
        |  XOR mask + SCAN marker
        v
response 0xb791f3dd

Result: PASS
```

OUTCOME **Project-owned code crosses the TrustZone boundary.**

BOUNDARY Communication gate passed; CAN hardware is Step 11.